Production-Ready Application Specification & Prompting Guide

> ***System Prompt Instruction for the AI Agent:*** *Use this document as a set of non-negotiable architectural requirements and constraints. When writing, generating, or modifying code, you must execute every task according to these implementation standards. Do not write bare prototype code; build for real-world resilience, scalability, and strict system security.*

Core Architectural & Pre-Deployment Pillars

1. Database Resilience & Scaling Architecture

Objective: Eliminate execution errors, prevent connection starvation, and secure query speeds under high horizontal read/write traffic volumes.

Implementation Guidelines for the Agent:

- **Database Indexing Strategy:** Audit all database schemas. Identify columns frequently parsed in `WHERE` clauses, `JOIN` conditions, or `ORDER BY` sorting metrics (e.g., `email`, `uuid`, `created_at`) and build explicit database indexes for them.
- **Connection Pooling Config:** Configure a definitive database connection pool (e.g., via Prisma, TypeORM, or native pool drivers). Enforce maximum connection limits (`max: 20-50` depending on database capacity) and implement timeout thresholds to clean up dead or zombie idle processes.
- **Declarative Migrations:** Do not use destructive syncing mechanisms (like `sync({ force: true })`) in production. Generate evolutionary data migration files using automated tracking frameworks to alter live environments safely.

2. Enterprise-Grade Authentication & Session Security

Objective: Prevent token interception, credential leaks, session hijacks, and unauthorized horizontal privilege escalations.

Implementation Guidelines for the Agent:

- **Token Expiration Paradigms:** Implement split-token workflows. Issue short-lived JSON Web Tokens (JWT) for access (e.g., 15 minutes) coupled with robust, rotation-enabled refresh tokens stored tightly within the system infrastructure.
- **Secure Cookie Policies:** All session identifiers or state cookies delivered via web responses must rigidly apply the following secure configurations: `HttpOnly=true`, `Secure=true`, and `SameSite=Strict`.
- **Cryptographic Passwords:** Enforce asynchronous hashing protocols on all passwords before saving data. Use **bcrypt** with a minimum work factor (salt rounds) of 10–12, or use modern **Argon2id** configurations.

3. Input Validation & Fault-Tolerant Error Isolation

Objective: Neutralize logical edge cases, clean unexpected inputs, and ensure data mutations fail gracefully without server crashes.

Implementation Guidelines for the Agent:

- **Sanitization & Runtime Schemas:** Implement compile-time and runtime validation layers (using schemas like Zod, Joi, or Pydantic). Sanitize string inputs to neutralize cross-site scripting (XSS) and SQL injection attempts.
- **Global Try-Catch Orchestration:** Wrap all asynchronous routing code and network transactions in structural error catchers. Intercept system faults to log structural stack traces privately on the server while returning user-safe status payloads.

4. Strict Regression Isolation & Testing Requirements

Objective: Guarantee that new code additions or feature iterations do not structurally alter or degrade existing live operational business features.

Implementation Guidelines for the Agent:

- **Automated Test Integration:** Build a comprehensive unit and integration test infrastructure (e.g., Jest, PyTest, Playwright). Write automated tests for all business logic functions to run consistently before compilation steps.
- **Isolated Side-Effect Review:** Before applying any structural modification to a source file, cross-check its internal imports and dependents to ensure the modification remains scoped and context-isolated.

5. Production Deployment Standardization & Optimization

Objective: Eliminate environment drifting mismatches between development environments and remote cloud infrastructures.

Implementation Guidelines for the Agent:

- **Immutable Runtime Containers:** Author a concrete production-grade multi-stage `Dockerfile`. Standardize target base engine runtimes, lock dependencies using strict lockfiles, and strip out developer tools to minimize image weights.
- **Strict Environment Variable Enforcements:** Abstract all database URLs, API secrets, tokens, and system parameters away from source control. Enforce structured configuration objects that fail early at boot if required variables are missing.
- **Optimized Production Builds:** Run comprehensive compilation workflows before server deployments to optimize code, shake dead branches, compress assets, and bundle performance builds.

6. Edge Security, Auditing & API Protection

Objective: Establish perimeter defense layers around APIs and track operational performance abnormalities automatically.

Implementation Guidelines for the Agent:

- **Dependency Scanner Integration:** Integrate automated software scanners to routinely parse third-party packages for outdated systems or active vulnerability exposures.
- **CORS and Security Headers:** Restrict cross-origin resource permissions (CORS) strictly to specific, verified production top-level domains. Implement core Helmet headers like Content Security Policy (CSP).
- **Traffic Rate Limiting:** Enforce global IP address limits on accessible endpoint routers (e.g., a maximum of 100 API queries per 15-minute interval per IP address) to counter spam attacks.